i am developing a Angular project using github copilot. Now i would like to implement ""AI-Augmented Code Review". How to do this. Give step by step instructions.

Nice, this is a solid upgrade from just *using Copilot* → actually *showcasing AI in your SDLC*. Let's make this practical and something you can demo.

🧠 What "AI-Augmented Code Review" actually means (in your case)

You're not replacing human review—you're **adding an AI reviewer layer** that:

- Reviews PRs automatically
- Flags bugs, security issues, bad practices
- Suggests improvements (Angular-specific too)
- Enforces coding standards

Think of it like:
👉 *Copilot writes code → AI reviewer critiques it*

🏗️ Architecture (simple but powerful)

```
Developer → GitHub PR → AI Review (LLM) → Comments on PR
                                  ↓
                        Optional: Auto-fix suggestions
```

⚙️ Step-by-Step Implementation

✅ Step 1: Create a Pull Request workflow

AI review should run **on PR creation or update**.

Create this file:

```
.github/workflows/ai-code-review.yml
```

✅ Step 2: Setup GitHub Action

Use GitHub Actions to trigger AI review.

YAML

```yaml
name: AI Code Review

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  ai-review:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Get PR diff
        run: |
          git fetch origin ${{ github.base_ref }}
          git diff origin/${{ github.base_ref }} > diff.txt

      - name: Run AI Review
        run: |
          python ai_review.py
```

✅ Step 3: Write AI Review Script

Create:

```
ai_review.py
```

Example using OpenAI API:

Python

Run

```python
import os
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

with open("diff.txt", "r") as f:
    diff = f.read()
```

```
prompt = f"""
You are a senior Angular reviewer.

Review the following code diff and provide:
1. Bugs
2. Performance issues
3. Security concerns
4. Angular best practices violations
5. Suggested improvements

Code:
{diff}
"""

response = client.chat.completions.create(
    model="gpt-5",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.2
)

review = response.choices[0].message.content

print(review)

# Save output
with open("review.txt", "w") as f:
    f.write(review)
```

✅ Step 4: Post Review Back to PR

Add this step to your workflow:

YAML

```yaml
      - name: Post Review Comment
        uses: actions/github-script@v7
        with:
          script: |
            const fs = require('fs');
            const review = fs.readFileSync('review.txt', 'utf8');

            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: review
            });
```

✅ Step 5: Add Secrets

In GitHub:

```
Settings → Secrets → Actions → New Secret
```

Add:

```
OPENAI_API_KEY = your_key
```

✅ Step 6: Make It Angular-Smart 🔥

Improve your prompt:

```
Focus on:
- Angular lifecycle misuse
- Change detection issues
- RxJS anti-patterns
- Memory leaks (subscriptions)
- Module structure
- Lazy loading
- Performance (ngFor, trackBy)
```

✅ Step 7: Add Rule-Based Checks (Hybrid AI)

Combine AI with static tools:

- ESLint (Angular rules)
- Prettier
- SonarQube (optional)

Update workflow:

YAML

```yaml
- name: Run ESLint
  run: npm run lint
```

✅ Step 8: (Advanced) Inline Comments Instead of One Big Comment

Instead of dumping everything:

👉 Parse AI output → map to file/line → comment inline

For that:

- Ask AI to return JSON:

```JSON
[
  {
    "file": "app.component.ts",
    "line": 45,
    "comment": "Avoid subscribing without unsubscribe"
  }
]
```

Then use:

```JavaScript
github.rest.pulls.createReviewComment()
```

✅ Step 9: Add Auto-Fix Suggestions (Next Level)

Enhance prompt:

```
Also suggest improved code snippets where applicable.
```

Now your PR gets:

- Issue
- Fix suggestion

✅ Step 10: Demo Narrative (Very Important)

When showcasing, say:

> "We use GitHub Copilot for generation, and an LLM-based reviewer to validate code quality, enforce Angular best practices, and reduce human review effort."

🚀 Optional Tools (if you want shortcut)

Instead of building everything:

- GitHub Copilot PR Review (limited)
- CodeRabbit (AI PR reviews)
- ReviewGPT
- SonarQube + AI plugin

🧠 Pro Tips (from real-world usage)

- Keep **temperature low (0–0.3)** → consistent reviews
- Limit diff size (chunk if needed)
- Cache reviews to reduce cost
- Add “severity levels” (High/Medium/Low)

⚡ If you want to go one step further

You can turn this into a **multi-agent system**:

- Agent 1 → Code quality reviewer
- Agent 2 → Security reviewer
- Agent 3 → Angular expert
- Agent 4 → Performance optimizer

If you want, I can next help you:

✅ ↪ Convert this into a **LangGraph multi-agent reviewer**
✅ ↪ Add **inline PR comments implementation**
✅ Or ↪ build a **full demo architecture diagram + repo structure**